

Arduino und vvvv koppeln

Worum geht es in diesem Blatt?

In diesem Blatt koppeln Sie die beiden Werkzeuge der vorigen Blätter: Der Arduino liefert Messwerte, vvvv verarbeitet und zeigt sie an. Sie richten die Verbindung zunächst von Hand ein und wechseln dann auf das Firmata-Protokoll, das die Kopplung deutlich vereinfacht. Am Ende steht eine Anwendung, die die Temperatur einer Kaffeetasse sichtbar macht.

Was erwarten wir uns von der Praktikumsdokumentation/Protokoll?

- Beantworten der gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Zusätzlich machen Sie bitte Screenshots ihrer Patches/Programme sowie Fotos, die Ihre Lösungen in „Aktion“ zeigen (vvvv-Screenshots können Sie direkt in „vvvv“ komfortabel über die Tastenkombinationen „**STRG + 1**“ bzw. „**STRG + 2**“ erstellen); speichern der Patches nicht vergessen!
- **Eigene Interpretation** von Erkenntnissen des Experiments, Auffälligkeiten
- Welche **Probleme** sind Ihnen begegnet und wie haben Sie diese **gelöst**?

Wie?

- **Beantworten Sie zur Abgabe den Fragebogen/Quiz „Moodle-Abgabe Blatt 5“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie dazu Ihre** Antworten (Protokoll ggfs. in der Praktikumsstunde direkt im Angabe-Worddokument oder separatem Textdokument erstellen), **Screenshots, Fotos, etc. in die jeweiligen Antwortfelder.**
- **vvvv-Patches abspeichern und als Anhänge mitabgeben!**
- **ACHTUNG: Chromium-basierte Browser scheinen keine Bilder kopieren zu können, nutzen Sie in dem Fall einen anderen Browser, z.B. Firefox**

Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

In diesem Aufgabenblatt sehen wir uns an, wie man die Vielseitigkeit eines Arduino-Boards (Blatt 5) mit dem Komfort und der Mächtigkeit von „vvvv“ (Blatt 5) kombinieren kann. Damit ist es möglich, in kürzester Zeit Prototypen und Systeme zu entwickeln, die mit beinahe beliebigen Ein- und Ausgabegeräten (Sensoren und Aktuatoren) funktionieren und damit nahezu „unendliche“ Möglichkeiten für eigene kreative Projekte erschließen.

Bitte beachten Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Einführung in vvvv“, die die Basisinformation zu vvvv, Grundbegriffe, erste Programme, etc., in einer Schritt-für-Schritt Anleitung (Tutorial-Stil) erläutern. Zusätzlich sei auf folgende Ressourcen hingewiesen, die Ihnen den Umgang mit „vvvv“ erleichtern können:

- Tutorial (ähnlich wie jenes der Vorlesung): [vvvvBasics - Tutorial in simple steps \(English\).pdf](#)
- Übersicht über die Tastatur- und Mausbelegung: [Leaflet dont panic - The noobs guide to vvvv.pdf](#)

1) Das Arduino mit „vvvv“ sprechen lassen

Das Arduino-Board kann, wie wir in Blatt 5 gesehen haben, „Standalone“ verwendet werden, d.h. die Logik wird in der Arduino IDE entwickelt, kompiliert, und auf das Board gespielt. In der Folge arbeitet das Arduino-Board als autarkes System, d.h. nimmt Werte von Sensoren entgegen, führt die Bearbeitung durch, und gibt Feedback/Rückmeldung via Aktoren (LEDs, Motoren, Displays, etc.) an die Außenwelt.

In den folgenden Aufgaben verwenden wir das Arduino-Board als „Interface“ (Schnittstelle). Werte der an das Arduino-Board angeschlossenen Sensoren werden direkt an den Computer übertragen (hier: an das Programm „vvvv“) und dort weiterverarbeitet. Das Board kann ebenfalls von einem externen Programm (hier wiederum „vvvv“; natürlich können dafür auch andere Programme verwendet werden) Daten empfangen und diese an Aktoren weitergeben – das Arduino-Board hat dabei selbst keine Logik, sondern fungiert nur als Schnittstelle zwischen der physischen Welt (Sensoren, Aktoren) und der Software auf dem Computer – **die Logik wird komplett in „vvvv“ entwickelt/programmiert.**

Für die erste Aufgabe benötigen wir folgende Komponenten:

- PC mit installiertem „vvvv“
- Arduino UNO-Board, USB-Kabel
- Steckbrett („Breadboard“)
- Potentiometer/Drehregler, Größe 13mm, 1kOhm
- Externe LED, rot (2 Pins)

1.a) Wert eines Analogsensors (Potentiometer) in „vvvv“ darstellen

Wir werden hier den Wert eines Potentiometers/Drehreglers über das Arduino-Board an „vvvv“ übermitteln und dort in einer geeigneten Form darstellen. Bei dieser Art des „*Interfacings*“ müssen wir, bevor wir auf die Daten in „vvvv“ zugreifen können, zunächst die Konfiguration der Ein-/Ausgänge, die Art der Datenübertragung, etc. auf dem Arduino-Board festlegen.

Schritt 1: Wir entwickeln als erstes ein einfaches Arduino-Programm, das sich um die Kommunikation kümmert. Sie können dieses Programm per „*Copy & Paste*“ in die Arduino IDE übertragen (bzw. die Datei aus *Moodle* bzw. dem *Datenaustausch-Laufwerk* „P:“ laden), kompilieren, und auf das Arduino-Board uploaden. Ergänzende Erklärungen:

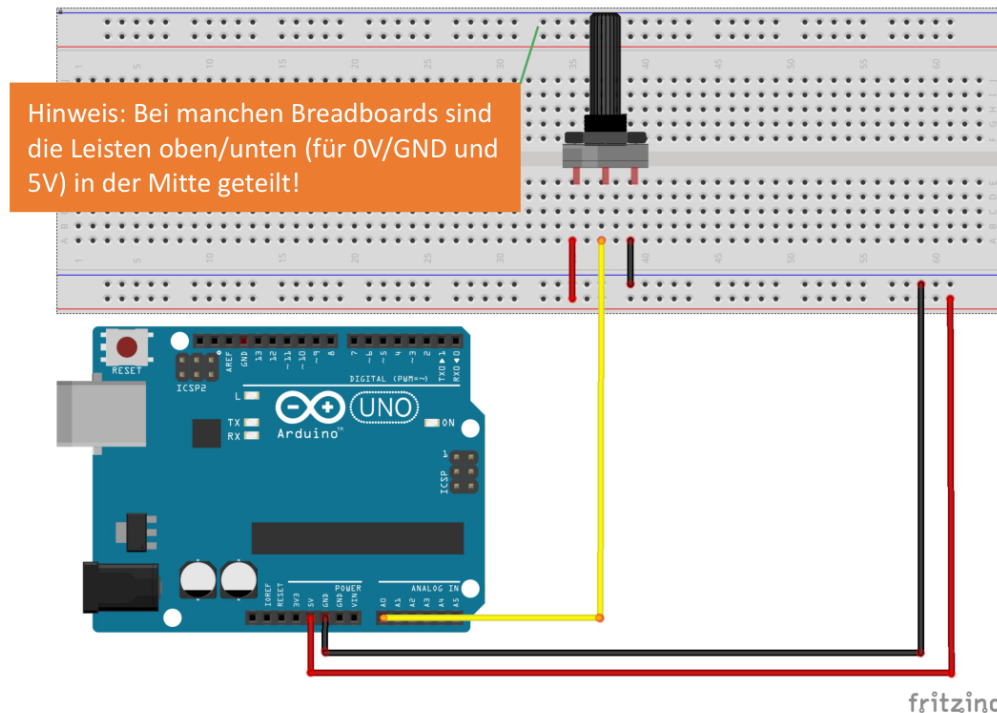
- Zur Kommunikation verwenden wir die serielle Schnittstelle (die wir in Blatt 5 bereits zu Debugging-Zwecken genutzt haben): `Serial.begin(9600)`; Die Baud-Rate (9600) gibt dabei an, wie viele Symbole pro Sekunde übertragen werden, 9600 reicht in den meisten Fällen aus
- Um den Inhalt der Variablen `sensorValue` auf die serielle Schnittstelle auszugeben, verwenden wir wiederum die Methode `Serial.println(sensorValue)`;

```
/*  
  Arduino als Interface für vvvv konfigurieren  
  
  modified October 15, 2021  
  by Andreas Riener, THI  
  */  
  
// Deklaration globaler Variablen  
int sensorPin = A0;  
int sensorValue = 0;
```

```
void setup() {  
  // Serielle Verbindung mit 9600 baud zur Kommunikation mit vvvv  
  Serial.begin(9600);  
}  
  
void loop() {  
  sensorValue = analogRead(sensorPin);  
  Serial.println(sensorValue);  
  delay(10);  
}
```



Schritt 2: Aufbau einer einfachen Schaltung wie nachfolgend dargestellt. Durch die Drehung des Reglers verändert sich dessen Widerstandswert. Der aktuelle Wert (entspricht der Position des Drehreglers) wird hier am analogen Eingang „A0“ des Arduino-Boards eingelesen.



Schritt 3: Als nächstes starten wir „vvvv“ und entwickeln die Logik, um Daten vom Arduino-Board zu empfangen.

Schritt 3: Als nächstes starten wir „vvvv“ und entwickeln die Logik, um Daten vom Arduino-Board zu empfangen. Das zentrale Element im „vvvv“-Patch ist dabei der „RS232 (Devices)“-Node. Dieser empfängt die Werte des Arduino-Boards auf der seriellen Schnittstelle. Für eine korrekte Kommunikation muss die sog. „Baud-Rate“, mit der das Arduino-Board sendet, gleich sein mit jener, die in „vvvv“ eingestellt ist. Nachfolgend sehen wir das „vvvv“-Programm, dass wir nun versuchen nachzubauen/selbst zu implementieren.

Vorgangsweise

- Starten Sie „vvvv“ und legen Sie einen neuen, leeren Patch an
- Nun fügen Sie nach und nach die einzelnen Komponenten („Nodes“) wie folgt ein (**siehe nachfolgende Skizze!**):
- **Hinweis: Die Nummernangaben in Klammern entsprechen den jeweiligen Zahlen in den Kreisen der Patch-Screenshots**

(1) Doppelklick (links) in das Fenster öffnet den „Node-Inspector“ – hier sind alle verfügbaren „Nodes“ (Komponenten) aufgelistet, über das Eingabefeld kann gefiltert werden. Suchen Sie hier den „RS232 (Devices)“-Node und fügen ihn in Ihren Patch ein. Vergrößern Sie den Node damit Sie besser dessen Optionen erkennen können bzw. leichter Verbindungen herstellen können

- **Hinweis (Erinnerung):** Für jeden Node gibt es im „Node-Inspector“ eine **Hilfeseite**. **Klick mit der rechten Maustaste auf den Node** im „Node-Inspector“ öffnet die zugehörige Hilfeseite bzw. direkt im **Patch: Node markieren und F1-Taste**
- **Hinweis (Erinnerung) 2:** Für eine leichtere Konfigurierbarkeit der einzelnen Nodes öffnen Sie den „**Herrn Inspektor**“ („vvvv“-Hauptmenü über mittlere Maustaste öffnen, dort im linken, unteren Quadranten „*Inspektor*“ wählen)
- **Hinweis (Erinnerung) 3:** Weißer Text auf dunklem Hintergrund stellt einen Beschreibungstext dar, den Sie zu jedem „Node“ als Kommentar angeben können (→ über den „Herr Inspektor“, Attribut „Descriptive Name“; hat keine Funktion sondern dient nur zur Information)



Weitere Elemente („Nodes“) fügen Sie mit einem Doppelklick links in das „vvvv“-Programmfenster („Patch“) ein; Konfiguration inkl. Datentypen der einzelnen Elemente:

(2) Node-Typ „IOBox (Value Advanced)“, im Inspektor als Value Type „Integer“ auswählen, als Y Input Value „9600“ eingeben

(3) Node-Typ „IOBox (Value Advanced)“, Value Type „Boolean“, Behavior „Bang“

(4) Node-Typ „IOBox (Enumerations)“; wenn Sie diesen Node mit dem „RS232“-Node verbinden (Parameter: Port Name), dann wird dieser Node mit den möglichen Optionen korrekt befüllt

(5) Node-Typ „IOBox (Value Advanced)“, Value Type „Boolean“, Behavior „Toggle“, Default „0“

(6) Node-Typ „IOBox (Enumerations)“; wenn Sie diesen Node mit dem „As String“-Node (7) verbinden (Parameter: Encoding), dann wird der Node korrekt befüllt

(7) Node-Typ „AsString (Raw)“, Encoding „us-ascii“ (eingestellt über (6))

(8) Node-Typ „IOBox (String)“

Hinweis: Der „RS232“-Node liefert die Werte als sogenannte „Byte-Sequenz“. Um diese „lesbar“ zu machen, wird dieser Datenstrom mit den Nodes 6+7 in einen ASCII-String umgewandelt und im Node (8) angezeigt

- Patch als „Potentiometer an vvvv basic.v4p“ abspeichern und erst **DANACH** mit Rechtsklick auf (5) „RS232 ein/aus“ das Programm (den Patch) starten. Sie sollten nun im Ausgabefeld (8) den Wert des Potentiometers sehen können

Frage 1: Fällt Ihnen etwas an der Ausgabe auf - was? Was könnte der Grund für dieses Verhalten sein (interpretieren Sie textuell)?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

- Um die in der Basisvariante des Programms/der Schaltung identifizierten Probleme besser verstehen zu können, sehen Sie sich nun folgende, verbesserte Variante der Schaltung an. Den fertigen Patch finden Sie als „Potentiometer an vvvv complete.v4p“ im Laufwerk „P:“ (Datenaustausch) bzw. *Moodle*. Experimentieren Sie mit der Schaltung und versuchen Sie zu verstehen, was die erweiterte Version macht – es wurden mehrere Verbesserungen eingebaut...



Frage 2: Beschreiben Sie (in eigenen Worten) was die erweiterte Version genau macht. An welchen Stellen wurden Verbesserungen vorgenommen? Welche? Wie wirken sich diese im Programm aus? Einige Tipps dazu:

1. Sie können dazu gerne einzelne Teile der Schaltung entfernen, z. B. den „Tokenizer“- oder „Clean“-Node herausnehmen bzw. die Konfiguration ändern...
2. Sehen Sie sich die Hilfeseiten der einzelnen Komponenten/Nodes an!

Vergessen Sie nicht auf die Dokumentation: Screenshots, Fotos Ihrer Schaltungen, v4-Patches bitte in das Protokoll einfügen ;-)

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

2) Vereinfachte Interaktion mittels der „Firmata“

Das Arduino-Board kann, wie wir in Blatt 5 gesehen haben, „Standalone“ verwendet werden, d.h. die Logik wird in der Arduino IDE entwickelt, kompiliert, und auf das Board gespielt. In der Folge arbeitet das Board autonom, d.h. kommt ohne Computer aus. Für interaktive Anwendungen ist das natürlich nicht geeignet. Wie wir in Aufgabe 1 (oben) gesehen haben, können wir jedoch auch Werte von Sensoren über das Arduino-Board an „vvvv“ übermitteln, in der Standardversion (mittels „RS232 (Devices)“-Node) jedoch nicht besonders flexibel (neue Projekte bzw. geringfügige Änderungen erfordern jeweils eine Anpassung der Konfiguration des Arduino-Boards).

Mithilfe des Firmata-Sketches und einem speziellen „FirmataBoard“-Node kann ein Arduino-Board (in „vvvv“) jedoch als flexibles und sehr mächtiges Interface betrieben werden. Wie das geht, sehen wir uns in der nächsten Aufgabe an.

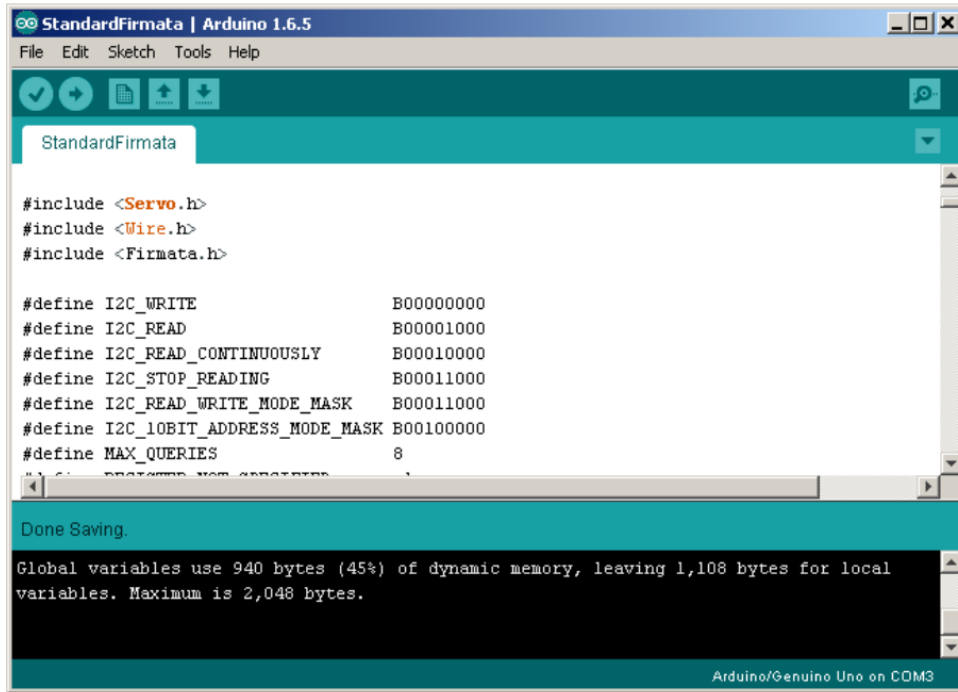
Was ist die Firmata eigentlich?

Die „Firmata“ ist ein generisches Protokoll, das die Kommunikation zwischen Mikrocontrollern und einer beliebigen Software auf dem Computer ermöglicht. Normalerweise müssten nämlich hierfür eigene Programme auf dem Arduino-Board geschrieben werden, die genaue Angaben über die angeschlossenen Pins und die verwendeten Funktionen machen. Außerdem muss sichergestellt werden, dass das Datenpaket auf der Empfängerseite richtig dekodiert wird, um mit den empfangenen Werten auch arbeiten zu können, etc.

Durch die „Firmata“ können wir nun bequem auf Sensoren und Aktoren des Mikrocontrollers zugreifen, ohne uns Gedanken über die Kommunikation und die Codierung machen zu müssen.

Vorbereitung: Firmata auf das Arduino-Board laden

Der große Vorteil des „FirmataBoard“-Nodes in „vvvv“ ist es, dass Sie sich bei Verwendung der „Firmata“ nicht mehr um die Kommunikation zwischen Arduino-Board und „vvvv“ kümmern müssen. Laden Sie die Standard-Firmata (Datei „StandardFirmata.ino“ steht zum Download in Moodle bzw. im Datenaustausch-Laufwerk „P:“ bereit) über die Arduino IDE auf das Arduino UNO-Board (Arduino IDE starten, „Firmata“-Datei über „File“ → „Open“ öffnen und über „Sketch“ → „Upload“ auf das Arduino-Board übertragen). Ab sofort können Sie das Board bequem von „vvvv“ aus ansteuern bzw. quasi „fernsteuern“.



```
#include <Servo.h>
#include <Wire.h>
#include <Firmata.h>

#define I2C_WRITE          B00000000
#define I2C_READ           B00001000
#define I2C_READ_CONTINUOUSLY B00010000
#define I2C_STOP_READING  B00011000
#define I2C_READ_WRITE_MODE_MASK B00011000
#define I2C_10BIT_ADDRESS_MODE_MASK B00100000
#define MAX_QUERIES        8
```

Done Saving.

Global variables use 940 bytes (45%) of dynamic memory, leaving 1,108 bytes for local variables. Maximum is 2,048 bytes.

Arduino/Genuino Uno on COM3

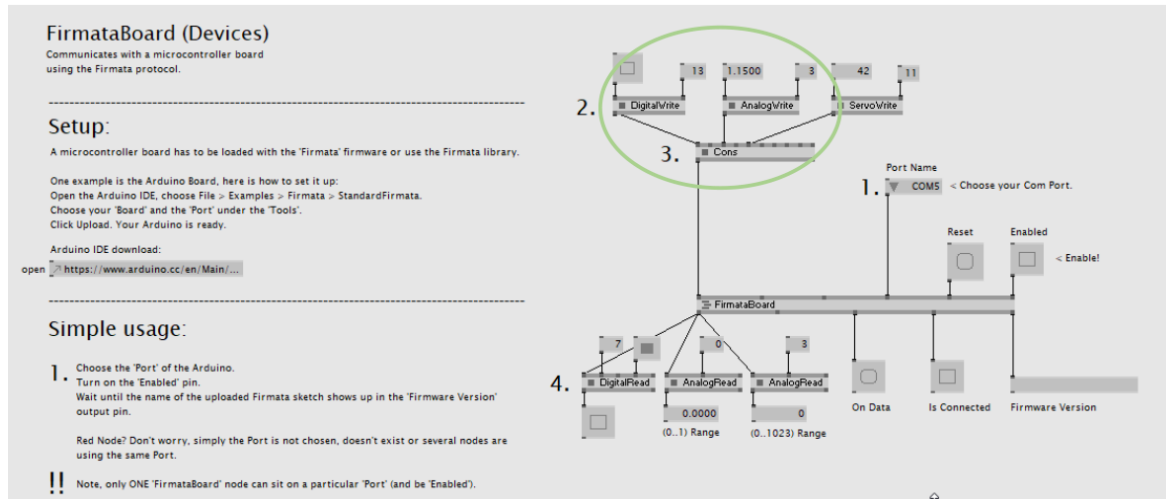
In der Folge sehen wir uns einige Schaltungen an, die wir bereits beim Arduino-Aufgabenblatt (mittels Programmierung des Arduino-Boards) gelöst haben. **Nun erfolgt die Implementierung der Logik ausschließlich über „vvvv“ – am Arduino-Board selbst (bzw. im Arduino-Code) sind keine Änderungen mehr vorzunehmen!**

Wir vergrößern den „FirmataBoard (Devices)“-Node, löschen die Nodes „Baud-Rate“, „AsString“, und die beiden „IOBox“ („us-ascii“, leere Box ganz unten), und hängen die Nodes „Portliste aktualisieren“, „Portname“, „Enabled“ an den „FirmataBoard“-Node an den richtigen Stellen ein (wenn Sie die Maus über die kleinen Quadrate am „FirmataBoard“-Node bewegen, sehen Sie welche Funktionalität damit verknüpft ist). Damit es übersichtlicher aussieht, können wir die einzelnen Nodes noch verschieben.

Mit Hilfe des „Cons (Firmata)“-Nodes fassen wir AnalogWrite, DigitalWrite des Arduino-Boards zusammen und verbinden dieses mit dem 1. Eingangspin des „FirmataBoard (Devices)“. **Das Ergebnis sehen Sie auf der Abbildung auf der nächsten Seite** – ihr Zwischenstand sollte etwa gleich aussehen ;-)



Nun können die zu verwendenden Pins des Arduinos definiert werden, so wie wir es auch in Arduino tun würden bzw. getan haben. Dafür gibt es „AnalogWrite“- und „DigitalWrite“-Nodes, die mittels „Cons (Firmata)“-Node konkateniert und an das „FirmataBoard (Devices)“ angeschlossen werden müssen. Sehen Sie sich dazu die Hilfe zum „FirmataBoard (Devices)“ bzw. „Cons (Firmata)“-Node an:



Frage 3: Wie bringen Sie nun eine LED an der Pin 9 zum Leuchten (einfaches Ein- und Ausschalten mit Hilfe einer IOBox in v4)? Bauen Sie die Schaltung auf dem Breadboard auf und erweitern Sie den „vvvv“-Patch um die notwendigen Nodes (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 4: Nun möchten wir eine weitere LED mit Hilfe von v4 unabhängig von der ersten LED „dimmen“ (Helligkeitswert analog steuern). Wie müssen Sie Ihre Arduino-Schaltung nun ergänzen? Welchen Arduino-Pin verwenden Sie für die 2. LED? Wie müssen Sie den „vvvv“-Patch verändern (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

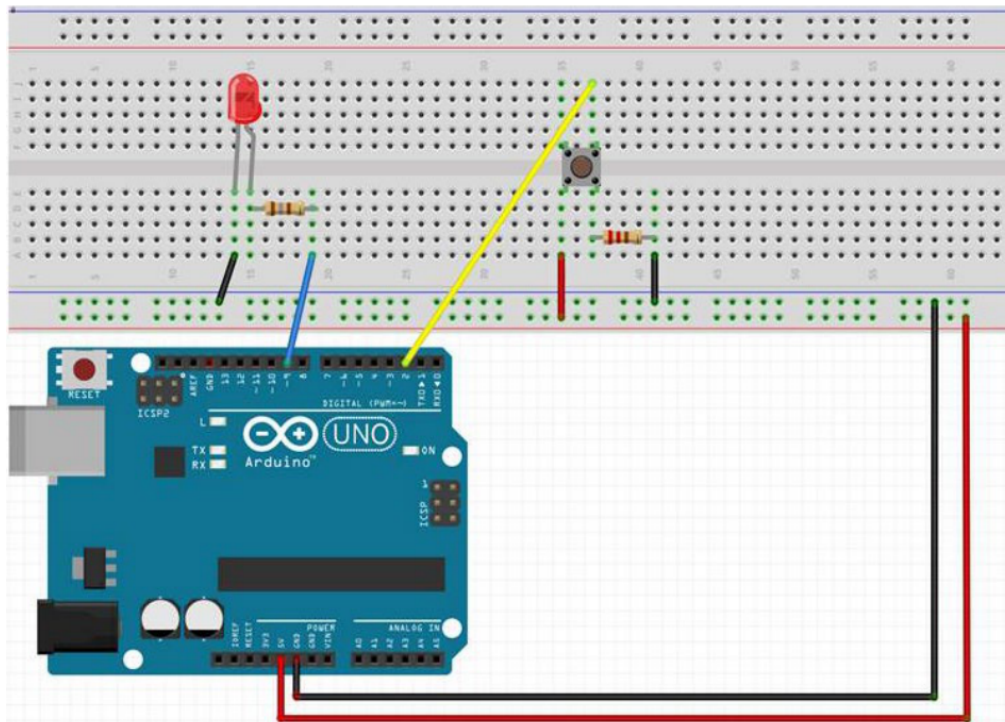
Hinweis zu „vvvv“ und Arduino über Firmata: Nachdem im Hintergrund natürlich Konfigurationen zwischen „vvvv“ und dem Arduino-Board über die serielle (USB-) Schnittstelle auszutauschen sind, **kann es bis zu 10 Sekunden dauern, bis ein „vvvv“-Patch auch tatsächlich mit dem Arduino-Board kommuniziert.** Planen Sie diese Wartezeit bei Ihren Tests ein (also: seien Sie nicht „zu“ ungeduldig...). Manchmal muss man auch die Portliste aktualisieren, damit die Kommunikation funktioniert.

ACHTUNG: Es darf immer nur ein Patch mit aktiviertem Firmata-Node offen/aktiv sein; ansonsten gegenseitige Beeinflussung und ggfs. seltsames Verhalten...

2.b) Verwenden eines „Buttons“ als Schalter für die LED

Und weiter geht's mit Ein/Ausgabe und Feedback: Der „Push-Button“ soll nun die LED ein-bzw. ausschalten (die „Standalone“-Version kennen wir ebenfalls schon aus dem Arduino-Blatt 5).

Schritt 1: Wir ergänzen zunächst die Schaltung am „Breadboard“ um den „Push-Button“ inkl. Verkabelung wie nachfolgend dargestellt („Push-Button“ an Eingang 2, LED an Eingangs-Pin 9, Vorwiderstand 10kOhm).



Schritt 2: Versuchen Sie nun selbst die die Logik in „vvvv“ so zu ergänzen, dass die LED binär ein-/ausgeschaltet wird (somit nicht mehr über den Slider, sondern über das Signal Ein/Aus, on/off, gedrückt/nicht gedrückt des Tasters an Pin 2 des „Arduino“-Boards). Speichern Sie den vorherigen Patch zunächst (d.h. bevor Sie Änderungen vornehmen) als „Firmata LED PushButton.v4p“ ab. Für die Änderung müssen Sie noch folgendes Wissen bzw. berücksichtigen:

- Analog zu „DigitalWrite“ gibt es auch einen „DigitalRead“-Node, der Ihnen den Wert des jeweiligen Pins des „Arduino“-Boards liefert. Diese muss an den „Pin State“-Pin des FirmataBoards angehängt werden. Versuchen Sie zunächst, den Status des Schalters (on/off) in einer „IOBox (Value Advanced)“ anzuzeigen...
- Zur Implementierung von Feedback (d.h. Arduino-Board Eingang steuert Verhalten an einem Ausgang des Arduino-Boards), müssen Sie zur Einhaltung von zeitlichen Constraints in den Feedback-Kreis einen „FrameDelay“-Node einfügen (**siehe Vorlesungs-Unterlagen/Tutorial**).

Frage 5: Wie sieht Ihre Schaltung aus? Funktioniert die Schaltung wie gewünscht? Was macht der „FrameDelay“-Node genau und was passiert falls Sie diesen nicht verwenden? Wo hatten Sie Probleme, etc.? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

2.c) Verwenden eines „Drehreglers“ zum Steuern der LED-Helligkeit

Als letzten Schritt möchten wir nun einen Drehregler (Potentiometer) verwenden, um die Helligkeit der LED zu regeln. Speichern Sie den vorherigen Patch zunächst (d.h. bevor Sie Änderungen vornehmen) als „Firmata LED Potentionmeter.v4p“ ab.

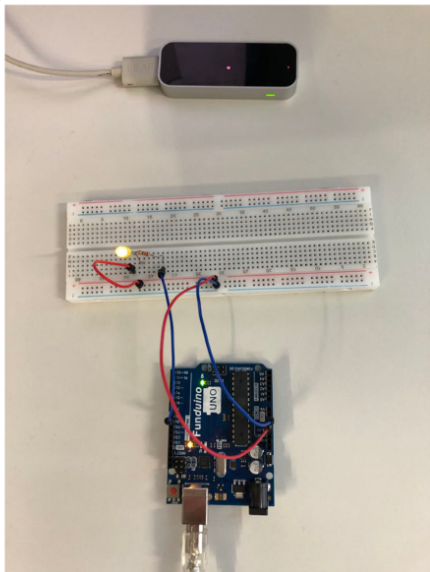
Schritt 1: Änderung des Aufbaus am Breadboard (der Taster inkl. Kabel kann abgebaut werden), der Drehregler ist zu ergänzen und integrieren. Dieser soll an Port 3 bzw. A3 des Arduino-Boards angeschlossen (und über einen „AnalogRead“-Node in vvvv ausgelesen) werden. Die linken und rechten Pins des Drehreglers sind dazu in der Schaltung auf 0V (GND) und 5V zu legen, der mittlere Pin liefert dann abhängig von der Stellung des Reglers einen Wert von 0...5 Volt (in vvvv: 0...1 als Realzahl).

Schritt 2: Ändern/ergänzen Sie nun in „vvvv“ die Programmlogik so, dass der Wert (die Stellung) des Drehreglers die Helligkeit der LED kontinuierlich regelt (ähnlich zu Aufgabe 2.a). Verwenden Sie einen „AnalogRead“-Node (Drehregler an Eingabeindex 3) und versuchen Sie zunächst, den Wert des Drehreglers in einer „IOBox (Value Advanced)“ anzuzeigen. Wenn das funktioniert, komplettieren Sie die Schaltung, sodass dieser Wert die LED-Helligkeit regelt. Dazu verwenden Sie einen „FrameDelay“- und dahinter einen „AnalogWrite“-Node (Ausgabe an Index/Pin 9). Hinweis: AnalogWrite erwartet als Input entweder 0...1 (Real) oder 0...1023 (Integer).

Frage 6: Wie sieht Ihre Schaltung aus? Funktioniert die Schaltung wie gewünscht? Wo hatten Sie Probleme, etc.? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

...setzen wir im Rahmen des Praktikums nicht um; wir möchten nur demonstrieren, wie einfach sich anstatt der Eingabe über Arduino/Firmata andere Eingabegeräte zur Steuerung via Arduino einsetzen lassen:



3) Eine „sinnvolle Anwendung“: Temperaturanzeige für die Kaffeemaschine

Über die Temperatur können Sie eine Vielzahl unterschiedlicher Informationen ermitteln. Mit einem Temperatursensor können Sie diese Daten als Eingangssignal im Computer erfassen. Im abschließenden Beispiel dieses Aufgabenblattes wird der eingelesene Wert eines Temperatursensors in vvvv verarbeitet und an eine RGB-LED gesendet, um die Temperatur eines Heißgetränks zu visualisieren.

3.a) Temperatursensor auslesen

Schritt 1: Modifizieren der Schaltung auf dem Steckbrett („Breadboard“).

Zuerst muss das Signal des Temperatursensors in vvvv empfangen werden. Auf dem Arduino-Board muss dazu wieder die „StandardFirmata“ geladen sein (falls noch nicht vorhanden). In diesem Aufbau wird ein Temperatursensor Typ LM35 verwendet (erinnern Sie sich an Vorführaufgabe 2 – hier hatten wir den LM35 zur Verwendung in einer Heizungssteuerung eingesetzt. Es handelt sich hier um den gleichen Sensor.)

Die Belegung können Sie dem Datenblatt (<http://www.ti.com/lit/ds/symlink/lm35.pdf>) entnehmen bzw. hier nachlesen:

- Temperaturbereich: -55 bis 150° Celsius (mit) bzw. **+2 bis 150° Celsius (ohne Pull-down Widerstand)**
- LM 35 liefert 0 V bei + 2°C
- Empfindlichkeit: 10 mV/° Celsius
- Anschlussschema siehe nachfolgend

Temperaturcharakteristik bei Verwendung im Arduino-Board (mit 5V Versorgungsspannung) und Standardbeschaltung (somit +2 bis 150° Celsius): (diese ist bei Verwendung im aktuellen Beispiel dann entsprechend zu berücksichtigen):

- 10mV pro °C und 0V bei +2°C → damit bei +150°C: $0,01V \cdot (150-2) = 1,48 V / 5V = 0,296$; der max. Ausgangswert des LM35 bei Versorgungsspannung von 5V ist somit **0,296**
- **Für die Temperaturermittlung können wir somit eine Map (0.0, 0.296, 2, 150) verwenden!**

Werteabschätzung z.B. für Raumtemperatur (22° Celsius):

- 10 mV/°C und 0V bei +2°C, d.h. $10 \text{ mV} \cdot (22-2) = 0,2V / 5 = 0,040$
- Bei **Raumtemperatur (22°C)** erwarten wir also etwa einen Wert des Temperatursensors von **0,040**.

Anschlussschema:

- (1) Der rechte Pin des Sensors wird mit dem 5V-Anschluss des Arduino-Boards verbunden
- (2) Der mittlere Pin mit dem „Analog-In“-Port 2
- (3) Der linke Pin mit „Ground“ (0V)



Schritt 2: Änderung/Erweiterung des Patches

Öffnen Sie den Patch „FirmataBoard TempSens.v4p“ aus dem Datenaustausch („P:“) und erweitern Sie diesen um folgenden Komponenten (siehe Screenshot auf der nächsten Seite):

(1) Simulation des Drehreglers durch einen „Slider“

Grundsätzlich verwenden wir auch hier wieder den Arduino-Node (mit der Firmata) zum Einlesen der Werte des Temperatursensors. Sollte kein Arduino-Board zur Verfügung stehen, können die Werte zwischen 0...1 (Real) mit einer entsprechenden „IOBox (Value Advanced)“ die als „Slider“ konfiguriert, ist (im „Herrn Inspektor“ Slider Behavior: „Slider“, Minimum „0.0“, Maximum „1.0“, Value Type „Real“), simuliert werden.

(2) Temperatursensor-Wert auslesen

Da der Temperatursensor an „Pin A2“ angeschlossen ist, muss der „AnalogRead“-Node den Index 2 eingestellt haben. Ist Ihre Schaltung korrekt müsste somit am Ausgangspin ein Wert zwischen 0...1 (Real) ankommen. Achtung: es kann **entweder** der Slider oder der Sensor verwendet werden!

(3) Größenkorrektur

Abhängig von der aktuellen Temperatur ist der im Renderer angezeigte Kreis ggfs. Sehr klein; über einen „* (Value)“-Node und einen Multiplikationsfaktor von z.B. „5“ kann die Größe auf einen Wunschwert eingestellt/korrigiert werden.

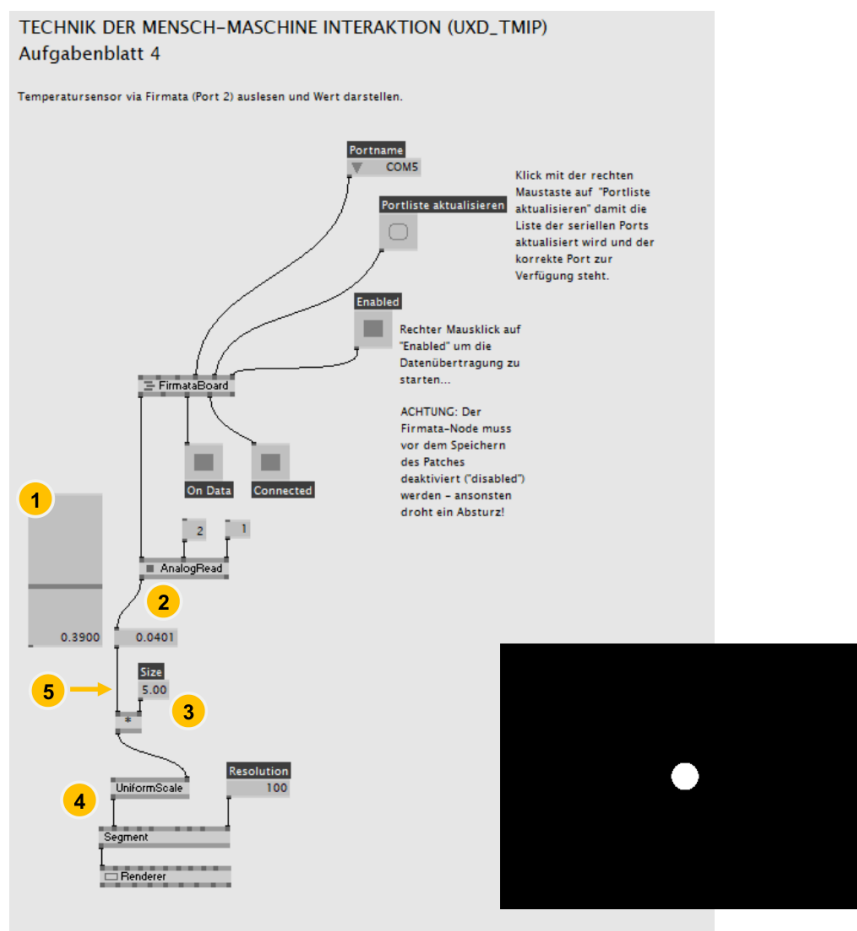
(4) Um den Messwert über die Größe eines Segments zu visualisieren muss ein „UniformScale (Transform)“ Node eingefügt werden. Das soll anschließend in einem „Renderer (EX9)“ Node angezeigt werden. Über den Wert von „Resolution“ können Sie einstellen, aus wie vielen Einzelsegmenten der Kreis zusammengesetzt sein soll.

Speichern Sie Ihren Patch als „FirmataBoard TempSens v2.v4p“ ab.

(5) Korrekte Temperatur ausgeben

Der Ausgang des Temperatursensors zeigt momentan noch einen schwer interpretierbaren Wert (von ca. 0,040) an. Versuchen Sie als nächsten – mit den oben angegebenen Informationen zur Werteabschätzung – den Wert in die korrekte Raumtemperatur umzurechnen und im Anschluss mit dem Temperaturwert die Anzeige des Kreissegments anzusteuern. Verwenden Sie dazu eine Map.

Speichern Sie Ihren Patch als „FirmataBoard TempSens final.v4p“ ab.



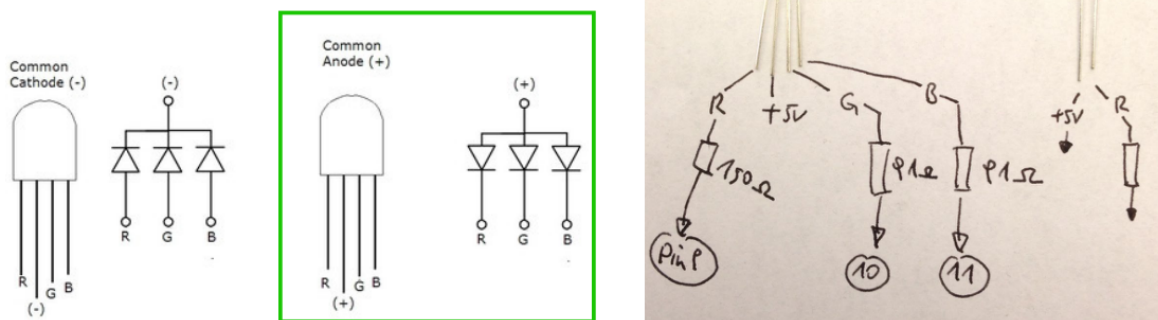
Frage 7: Wie musste der Patch erweitert werden, um die korrekte Temperatur anzuzeigen? Welche Erkenntnisse haben Sie gewonnen? Haben Sie dabei Probleme festgestellt? Welche?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

3.b) RGB-LED mit Temperatursensor steuern

Beachtung folgender Punkte:

- Die RGB-Leuchtdiode die wir verwenden verfügt über insgesamt 4 Pins. Je ein Pin für die 3 Farbkanäle (rot, grün und blau) sowie einen gemeinsamen Pin für die Versorgungsspannung 5 V. Die Bauweise einer solchen LED wird auch als „Common Anode“ (gemeinsame Anode) im Gegensatz zu „Common Cathode“ (gemeinsame Kathode) bezeichnet – siehe nachfolgende Skizze – und bedeutet, dass sich die 3 in der LED verbauten kleinen Einzeleuchtdioden den Pin für die Versorgungsspannung teilen. Der „Plus-Pin“ (Anode) ist dabei der längste Pin der LED, Pins für R und G sind etwa gleich lang und der Pin für Blau ist am Kürzesten (zeigt ebenfalls die Skizze).



Quelle: https://moderndevice.com/wp-content/uploads/2013/09/rgb_large.jpg Quelle: Prof. Dr. Andreas Riener

- Ermittlung der Vorwiderstände. Die Durchlass-/Sperrereigenschaften der verschiedenen Leuchtdioden (R, G, B) sind unterschiedlich, demnach sind auch unterschiedliche Vorwiderstände wie folgt zu verwenden (Berechnung z. B. auf <http://www.leds.de/Widerstandsrechner>):
- **Rot:** Durchlassspannung 2.1 V, Betriebsspannung 5 V, Betriebsstrom 20 mA; $(5\text{ V} - 2.1\text{ V}) / 20\text{ mA} = 145\ \Omega \rightarrow$ nächsthöherer verfügbarer Wert = **150 Ω** (Braun, Grün, Schwarz, Schwarz)
- **Grün:** Durchlassspannung 3.7 V, ergibt 65 $\Omega \rightarrow$ nächsthöherer verfügbarer Wert = **91 Ω** (Weiß, Braun, Schwarz, Gold)
- **Blau:** Durchlassspannung 3.1 V, ergibt 95 $\Omega \rightarrow$ nächster verfügbarer Wert = **91 Ω** (Weiß, Braun, Schwarz, Gold)

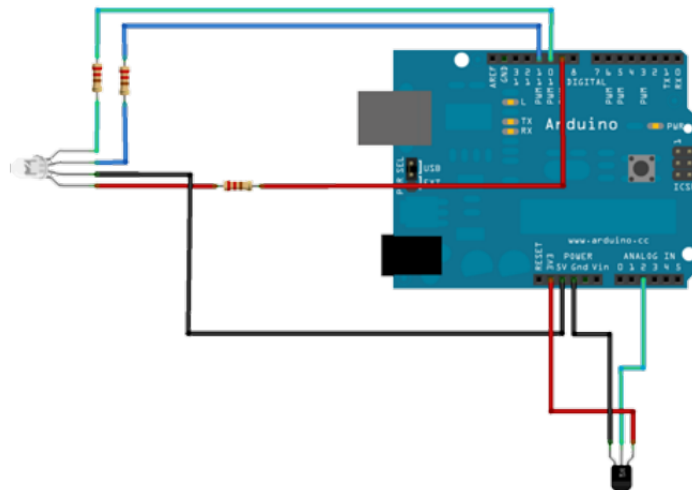
Schritt 2: Vorbereitung des Arduino-Boards für die Kommunikation mit „vvvv“. Nachdem wir die Standard-Firmata (Datei „StandardFirmata.ino“) bereits zuvor auf das Arduino-Board geladen haben, fällt dieser Schritt weg bzw. ist hier nichts zu tun – Juhu!

Schritt 3: Bauen Sie Ihre Schaltung entsprechend des nachfolgenden Schaltplans (Fritzing-Skizze) nach. Verwenden Sie dazu Ihr Breadboard.

Schritt 4: Versuchen Sie als nächstes, die rote, grüne, blaue LED in der RGB-LED über den Wert des Temperatursensors (real bzw. simuliert) anzusteuern und zum Leuchten zu bringen (über Pins 9, 10, 11).

Frage 8: Wie sieht Ihre Schaltung nun aus? Wie verhält sich die Schaltung (bei Simulation des Wert des Temperatursensors und bei tatsächlicher Verwendung des Temperatursensors)? Können Sie den Wert des „realen“ Temperatursensors beeinflussen/wie? (Abgabe Screenshot bzw. Foto des *.v4p-Patches sowie den .v4p-Patch als Datei).

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).



Schritt 5: Nun müssen wir noch die Steuerung für die LED in das v4-Programm einbauen. Erweitern Sie nun den Patch aus der Aufgabe 3.a. Speichern Sie den Patch unter „FirmataBoard TempSens RGB.v4p“ ab.

(1) Der Output des Sensors muss auf die Hue Werte eines gewählten Farbspektrums umgerechnet werden. Fügen Sie dafür den Node „Map (value)“ ein und verbinden Sie den Output des „TemperaturSensor“-Nodes mit dem Input-Pin.

(2) Geben Sie einen minimalen und maximalen Wert für den Sensor Output Wert an, sowie einen minimalen und maximalen Zielwert. Dafür müssen Sie jeweils eine „IOBox (Value Advanced)“ einfügen.

(3) Die Zielwerte bestimmen die Farbe der RGB-LED für die minimale und maximale aufgenommene Temperatur des Sensors. Mit dem Node „IOBox (Color)“ können Sie sich einen Farbraum anzeigen lassen, der zur Orientierung dient. Fügen Sie außerdem eine weitere „IOBox (Value Advanced)“ ein. Im Herrn Inspektor können Sie sich mit „Show Slider“ einen Slider anzeigen lassen um Werte zu definieren.

(4) Verbinden Sie die Ausgabe der „IOBox“ mit dem „Destination Minimum“ Pin. Mit dem Einfügen eines „HSV (Color Join)“-Nodes, ebenfalls am Ausgang, können Sie die Farbe errechnen und mit einer „IOBox (Color)“ visuell anzeigen lassen.

(5) Wiederholen Sie diesen Vorgang auch für den „Destination Maximum“-Pin. Nun haben wir alle nötigen Eingabewerte für den „Map (Value)“-Node definiert.

(6) Der Wert der nun durch den Temperatursensor aufgenommen wurde muss nun in einen tatsächlichen Farbwert umgewandelt werden. Dafür brauchen Sie, wie bereits oben, einen „HSV (Color Join)“-Node. Mit einer „IOBox (Color)“ kann dieser dann auch visuell im Patch angezeigt werden.

(7) Mit Hilfe des „RGB (Color Split)“-Nodes werden die Werte für den roten, grünen und blauen Kanal der LED ermittelt.

(8) Nun können Sie die einzelnen RGB-Werte den RGB-LED Pins übergeben.

(9) Jetzt können Sie den Output des „FrameDelay“ Nodes mit dem Input der Input-Pins der „AnalogWrite“-Nodes verbinden. Speichern Sie Ihren Patch als „FirmataBoard TempSens RGB.v4p“ ab.

Frage 9: Warum müssen die RGB LEDs an Pins mit PWM angeschlossen sein?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

Schritt 6: Zuletzt können wir testen, ob unsere Kopplung zwischen vvvv und Arduino auch funktioniert. Nehmen Sie sich eine Tasse und kochen Sie Wasser in dem Wasserkocher. Stellen Sie die heiße Tasse auf den Sensor und beobachten Sie was passiert.

Frage 10: Was beobachten Sie? Welches Problem stellen Sie fest? Beschreiben Sie die Veränderung Ihrer LED.

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

Frage 11: Wodurch lässt sich dieses Problem beheben und Ihre Temperaturanzeige noch verbessern?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).

Finale Schaltung:





BITTE BEACHTEN: Am Ende der Praktikumsstunde sind...

- ...die aufgebauten Schaltungen wieder abzubauen
- ...sämtliche Bauteile wieder sauber und vollständig in die Schachtel zu verpacken; Widerstände wieder in die richtigen Klebestreifen zu stecken, Drehregler/LEDs in die Säckchen zu geben, etc.
- ...die Arbeitsplätze sauber und gereinigt zu verlassen.